

Oracle Advanced Queuing (AQ)

Para implementar colas en Oracle usando Oracle Advanced Queuing (AQ), el flujo de trabajo estándar requiere definir el tipo de datos del mensaje, crear la tabla contenedora, crear la cola y finalmente activarla. A continuación, tienes un ejemplo práctico paso a paso para configurar y operar una cola de consumidor único.

1. Crear el tipo de objeto (Payload) Oracle AQ almacena los mensajes estructurados como objetos de la base de datos. Definiremos un tipo de objeto simple para enviar:

```
CREATE OR REPLACE TYPE tp_mensaje_notificacion AS OBJECT (  
    id_notificacion NUMBER,  
    destinatario     VARCHAR2(100),  
    mensaje          VARCHAR2(4000),  
    fecha_creacion   DATE  
);
```

2. Crear la tabla de la cola y la cola (DBMS_AQADM) Utilizamos el paquete de administración para crear la infraestructura física (la tabla) y la lógica (la cola).

```
BEGIN  
  
    -- 1. Crear la tabla que almacenará físicamente los mensajes  
    DBMS_AQADM.CREATE_QUEUE_TABLE (  
        queue_table      => 'tbl_cola_notificaciones',  
        queue_payload_type => 'tp_mensaje_notificacion'  
    );  
  
    -- 2. Crear la cola lógica asociada a la tabla anterior  
    DBMS_AQADM.CREATE_QUEUE (  
        queue_name => 'cola_notificaciones',  
        queue_table => 'tbl_cola_notificaciones'  
    );  
  
    -- 3. Iniciar la cola para permitir encolar (enqueue) y desencolar  
    (dequeue)  
    DBMS_AQADM.START_QUEUE (  
        queue_name => 'cola_notificaciones'  
    );  
END;  
/
```

3. Encolar un mensaje (DBMS_AQ) Para añadir un mensaje a la cola, utilizamos el paquete de operaciones DBMS_AQ.ENQUEUE. Cada mensaje requiere especificar propiedades de control opcionales.

```

DECLARE
    v_opciones_encolar    DBMS_AQ.ENQUEUE_OPTIONS_T;
    v_propiedades_msg     DBMS_AQ.MESSAGE_PROPERTIES_T;
    v_msg_id              RAW(16); -- Almacena el identificador único del
mensaje generado por Oracle
    v_payload             tp_mensaje_notificacion;
BEGIN

    -- Instanciar el objeto con los datos que queremos enviar
    v_payload := tp_mensaje_notificacion(
        id_notificacion => 101,
        destinatario    => 'usuario@correo.com',
        mensaje         => 'Tu código de verificación es 5543',
        fecha_creacion  => SYSDATE
    );

    -- Insertar el mensaje en la cola
    DBMS_AQ.ENQUEUE (
        queue_name        => 'cola_notificaciones',
        enqueue_options   => v_opciones_encolar,
        message_properties => v_propiedades_msg,
        payload           => v_payload,
        msgid             => v_msg_id
    );

    -- Confirmar la transacción (esencial para que el mensaje sea visible)
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Mensaje encolado con ID exitosamente.');
```

4. Desencolar un mensaje (DBMS_AQ) Para procesar y consumir el mensaje, ejecutamos un DEQUEUE. Por defecto, esta operación bloquea la ejecución (espera) hasta que aparezca un mensaje disponible si la cola está vacía.

```

DECLARE
    v_opciones_desencolar DBMS_AQ.DEQUEUE_OPTIONS_T;
    v_propiedades_msg     DBMS_AQ.MESSAGE_PROPERTIES_T;
    v_msg_id              RAW(16);
    v_payload             tp_mensaje_notificacion;
BEGIN
    -- Opcional: Configurar para que no se quede esperando indefinidamente si
está vacía
    v_opciones_desencolar.wait := DBMS_AQ.NO_WAIT;

    -- Extraer el primer mensaje disponible de la cola (First-In, First-Out)
    DBMS_AQ.DEQUEUE (
        queue_name        => 'cola_notificaciones',
```

```

        dequeue_options    => v_opciones_desencolar,
        message_properties => v_propiedades_msg,
        payload            => v_payload, -- Aquí se vierte el objeto extraído
        msgid              => v_msg_id
    );

    -- Mostrar los datos extraídos
    DBMS_OUTPUT.PUT_LINE('ID Notificación: ' || v_payload.id_notificacion);
    DBMS_OUTPUT.PUT_LINE('Destinatario:      ' || v_payload.destinatario);
    DBMS_OUTPUT.PUT_LINE('Mensaje:          ' || v_payload.mensaje);

    -- Confirmar la extracción (remueve definitivamente el mensaje de la cola)
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        -- Captura el error si la cola está vacía y usamos NO_WAIT
        IF SQLCODE = -25228 THEN
            DBMS_OUTPUT.PUT_LINE('No hay mensajes disponibles en la cola.');
```

```

        ELSE
            RAISE;
        END IF;
    END;
```

Detalles importantes a tener en cuenta

1. **Transaccionalidad:** Las operaciones ENQUEUE y DEQUEUE forman parte de la transacción de tu sesión. Si haces un ROLLBACK después de desencolar, el mensaje regresa automáticamente a la cola.
2. **Visibilidad:** Hasta que el productor no haga un COMMIT en el Paso 3, el consumidor del Paso 4 no podrá ver el mensaje.
3. **Purga automática:** Una vez procesado exitosamente el mensaje y ejecutado el COMMIT en el Paso 4, el registro se marca para ser eliminado de la tabla física de forma automática por los procesos internos de Oracle.

ADMINISTRACION DE EJECUCION

Para que puedas probar el ejemplo correctamente en tu base de datos, necesitas contar con ciertos privilegios del sistema y roles específicos de Oracle Advanced Queuing (AQ). Si intentas ejecutar el código con un usuario estándar sin estos permisos, el sistema arrojará errores de falta de privilegios.

A continuación, te detallo los pasos para asignar los permisos (como administrador) y cómo verificar que todo funcione.

1. **Otorgar Permisos (Ejecutar como Administrador / SYSDBA)** Conéctate con una cuenta administradora (por ejemplo, SYSTEM o SYS) y dale los siguientes accesos al usuario que va a realizar la prueba (reemplaza TU_USUARIO por el nombre real de tu esquema):

```
-- 1. Conceder el rol básico de administración de colas
GRANT AQ_ADMINISTRATOR_ROLE TO TU_USUARIO;

-- 2. Conceder el rol para ejecutar operaciones (encolar/desencolar)
GRANT AQ_USER_ROLE TO TU_USUARIO;

-- 3. Otorgar permiso explícito de ejecución sobre los paquetes
GRANT EXECUTE ON DBMS_AQADM TO TU_USUARIO;
GRANT EXECUTE ON DBMS_AQ TO TU_USUARIO;

-- 4. Otorgar cuota de espacio en el tablespace (necesario para crear la tabla de la cola)
ALTER USER TU_USUARIO QUOTA UNLIMITED ON USERS; -- 0 el tablespace que uses
```

PRUEBA

1. Preparar la Consola de PruebasConéctate ahora con TU_USUARIO. Antes de ejecutar los bloques PL/SQL que vimos anteriormente, asegúrate de activar la salida de texto en tu herramienta (SQL*Plus, SQL Developer, DBeaver, etc.) para que puedas ver los resultados de DBMS_OUTPUT:

```
SET SERVEROUTPUT ON;
```

2. Ejecución en OrdenSigue estrictamente este orden con los scripts del mensaje anterior:
 - Ejecuta el Paso 1 (Crear el tipo tp_mensaje_notificacion).
 - Ejecuta el Paso 2 (Bloque anónimo con DBMS_AQADM para crear la tabla, la cola e iniciarla).
 - Ejecuta el Paso 3 (Bloque para encolar el mensaje). Verás el texto en consola confirmando el éxito.
 - Ejecuta el Paso 4 (Bloque para desencolar el mensaje). Verás los datos del mensaje impresos en pantalla.

CONTROL DE EJECUCION

Si quieres comprobar "detrás de escena" qué está pasando con la infraestructura que acabas de crear, puedes consultar las vistas del diccionario de datos de Oracle con estas consultas:

```
-- Ver si la tabla de la cola existe y qué tipo de datos almacena
SELECT queue_table, object_type, recipients
FROM user_queue_tables;

-- Ver si la cola está activa para recibir y entregar mensajes
SELECT name, queue_table, enqueue_enabled, dequeue_enabled
FROM user_queues
WHERE name = 'COLA_NOTIFICACIONES';

-- Ver el contenido físico de la cola (mientras el mensaje no haya sido
```

```
desencolado con COMMIT)
SELECT msg_id, queue, state, user_data
FROM tblColaNotificaciones;
```

MULTIDIFUSION (UN MENSAJE PARA VARIOS CONSUMIDORES)

Para lograr que un mismo mensaje sea consumido por varios clientes o usuarios independientes, necesitas configurar una Cola de Múltiples Consumidores (Multiple Consumer Queue) basada en el patrón de diseño Publicador/Suscriptor (Pub/Sub).

En este modelo, el mensaje permanece en la cola hasta que todos los suscriptores registrados lo hayan desencolado. A continuación, te muestro los cambios que debes aplicar paso a paso:

1. Crear la Tabla de la Cola con Soporte Multiconsumidor: Al crear la tabla de la cola con DBMS_AQADM.CREATE_QUEUE_TABLE, debes encender explícitamente el parámetro multiple_consumers => TRUE. (**Nota:** Si usas el mismo usuario anterior, primero debes borrar o cambiar de nombre la cola previa para evitar conflictos).

```
BEGIN
-- 1. Crear la tabla especificando múltiple consumidor
DBMS_AQADM.CREATE_QUEUE_TABLE (
    queue_table      => 'tblColaDifusion',
    queue_payload_type => 'tpMensajeNotificacion',
    multiple_consumers => TRUE -- <-- ESTA ES LA CLAVE
);

-- 2. Crear la cola
DBMS_AQADM.CREATE_QUEUE (
    queue_name => 'colaDifusion',
    queue_table => 'tblColaDifusion'
);

-- 3. Iniciar la cola
DBMS_AQADM.START_QUEUE (
    queue_name => 'colaDifusion'
);
END;
```

2. Registrar a los Suscriptores (Clientes) Antes de enviar mensajes, debes registrar qué "clientes" están escuchando esta cola. Vamos a registrar dos suscriptores ficticios: CLIENTE_A y CLIENTE_B.

```
DECLARE
    v_suscriptor SYS.AQ$_AGENT;
BEGIN
-- Registrar el Primer Suscriptor
v_suscriptor := SYS.AQ$_AGENT('CLIENTE_A', NULL, NULL);
DBMS_AQADM.ADD_SUBSCRIBER(
```

```

        queue_name => 'cola_difusion',
        subscriber => v_suscriptor
    );

    -- Registrar el Segundo Suscriptor
    v_suscriptor := SYS.AQ$_AGENT('CLIENTE_B', NULL, NULL);
    DBMS_AQADM.ADD_SUBSCRIBER(
        queue_name => 'cola_difusion',
        subscriber => v_suscriptor
    );
END;

```

3. Encolar el Mensaje (Igual que antes) El proceso para meter un mensaje a la cola es exactamente idéntico al de consumidor único. Oracle se encargará internamente de saber que este mensaje va dirigido a todos los suscriptores activos.

```

DECLARE
    v_opciones_encolar    DBMS_AQ.ENQUEUE_OPTIONS_T;
    v_propiedades_msg     DBMS_AQ.MESSAGE_PROPERTIES_T;
    v_msg_id              RAW(16);
    v_payload              tp_mensaje_notificacion;
BEGIN
    v_payload := tp_mensaje_notificacion(
        id_notificacion => 202,
        destinatario    => 'todos@empresa.com',
        mensaje          => 'Mantenimiento programado de servidores a las
22:00.',
        fecha_creacion  => SYSDATE
    );

    DBMS_AQ.ENQUEUE (
        queue_name          => 'cola_difusion',
        enqueue_options     => v_opciones_encolar,
        message_properties  => v_propiedades_msg,
        payload              => v_payload,
        msgid                => v_msg_id
    );

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Mensaje de difusión enviado de forma exitosa.');
```

4. Desencolar identificándose como un cliente específico Al momento de consumir (DEQUEUE), el cliente debe identificarse usando su nombre en las opciones de desencolado (navigation o consumer_name). Si no se identifica, Oracle arrojará un error indicando que la cola requiere un nombre de consumidor.

- Prueba con el **CLIENTE_A**:

```

DECLARE
--
v_opciones_desencolar DBMS_AQ.DEQUEUE_OPTIONS_T;
v_propiedades_msg     DBMS_AQ.MESSAGE_PROPERTIES_T;
v_msg_id              RAW(16);
v_payload              tp_mensaje_notificacion;
--
BEGIN
--
-- INDICAR QUÉ CLIENTE ESTÁ CONSUMIENDO
v_opciones_desencolar.consumer_name := 'CLIENTE_A';
v_opciones_desencolar.wait          := DBMS_AQ.NO_WAIT;
--
DBMS_AQ.DEQUEUE (
    queue_name          => 'cola_difusion',
    dequeue_options     => v_opciones_desencolar,
    message_properties  => v_propiedades_msg,
    payload             => v_payload,
    msgid               => v_msg_id
);
--
DBMS_OUTPUT.PUT_LINE(
    'CLIENTE_A procesó el mensaje: ' || v_payload.mensaje
);
--
COMMIT;
--
END;

```

- Prueba con el **CLIENTE_B**:

Si ejecutas el bloque anterior cambiando la línea a `v_opciones_desencolar.consumer_name := 'CLIENTE_B';`, verás que el **CLIENTE_B** también recibe el mismo mensaje, a pesar de que el **CLIENTE_A** ya hizo COMMIT.

¿Cómo funciona la eliminación del mensaje? Si haces una consulta a la tabla física `SELECT * FROM tbl_cola_difusion;` verás que el registro sigue ahí mientras falte algún suscriptor por leerlo. Oracle AQ mantiene internamente un mapa de bits. Solo cuando tanto el **CLIENTE_A** como el **CLIENTE_B** hayan hecho su respectivo DEQUEUE y COMMIT, el mensaje se removerá por completo del sistema. ¿Te gustaría probar este modelo Pub/Sub en tu entorno, o te interesa saber cómo hacer que un mensaje vaya dirigido solo a un grupo selecto de suscriptores dinámicamente en lugar de a todos?

IMPLEMENTACIÓN CON LISTA DE DESTINATARIOS DINÁMICA

Para dirigir un mensaje de forma dinámica a un grupo selecto de suscriptores (sin que le llegue a todos los registrados en la cola), Oracle AQ ofrece dos estrategias nativas: Lista de Destinatarios Explícita (Recipient List): Defines los destinatarios exactos en el código en el momento de encolar (ENQUEUE).

Es la opción ideal cuando el emisor sabe exactamente a quién va dirigido el mensaje.

Reglas de Filtrado Condicional (Subscriber Rules): Los suscriptores se registran indicando una condición lógica basada en el contenido del mensaje (por ejemplo, "solo recibir si la prioridad es alta"). Oracle evalúa la regla en tiempo de ejecución.

A continuación, te explico detalladamente cómo implementar la primera opción (Lista de Destinatarios), que es la forma más directa y dinámica de segmentar los envíos.

No necesitas cambiar la configuración de la cola que creamos en el paso anterior (cola_difusion), ya que las colas de múltiples consumidores soportan esto de forma nativa. Asegúrate de tener registrados tus suscriptores en la cola (por ejemplo, CLIENTE_A, CLIENTE_B y añadiremos un CLIENTE_C para la prueba).

1. Registrar un tercer cliente de prueba Si no existe, añade un tercer suscriptor para ver cómo lo excluimos del envío:

```
DECLARE
--
v_suscriptor SYS.AQ$_AGENT;
--
BEGIN
--
v_suscriptor := SYS.AQ$_AGENT('CLIENTE_C', NULL, NULL);
DBMS_AQADM.ADD_SUBSCRIBER('cola_difusion', v_suscriptor);
--
END;
```

2. Encolar el mensaje especificando los destinatarios En el bloque de encolado, utilizaremos la propiedad recipient_list dentro del registro DBMS_AQ.MESSAGE_PROPERTIES_T. Crearemos un arreglo dinámico para indicarle a Oracle que este mensaje solo debe ser visible para el CLIENTE_A y el CLIENTE_C, ignorando al CLIENTE_B.

```
DECLARE
--
v_opciones_encolar DBMS_AQ.ENQUEUE_OPTIONS_T;
v_propiedades_msg DBMS_AQ.MESSAGE_PROPERTIES_T;
v_lista_destinatarios DBMS_AQ.AQ$_RECIPIENT_LIST_T; -- Arreglo de agentes
v_msg_id RAW(16);
v_payload tp_mensaje_notificacion;
--
BEGIN
--
-- 1. Definir los destinatarios dinámicamente para ESTE mensaje específico
v_lista_destinatarios(1) := SYS.AQ$_AGENT('CLIENTE_A', NULL, NULL);
v_lista_destinatarios(2) := SYS.AQ$_AGENT('CLIENTE_C', NULL, NULL);
--
-- 2. Asignar la lista a las propiedades del mensaje
```



```

v_propiedades_msg.recipient_list := v_lista_destinatarios;
--
-- 3. Crear el contenido del mensaje
v_payload := tp_mensaje_notificacion(
    id_notificacion => 303,
    destinatario    => 'Grupo Selecto',
    mensaje         => 'Mensaje exclusivo para Clientes A y C.',
    fecha_creacion  => SYSDATE
);
--
-- 4. Encolar
DBMS_AQ.ENQUEUE (
    queue_name      => 'cola_difusion',
    enqueue_options  => v_opciones_encolar,
    message_properties => v_propiedades_msg,
    payload          => v_payload,
    msgid            => v_msg_id
);
--
COMMIT;
--
DBMS_OUTPUT.PUT_LINE('Mensaje encolado para destinatarios específicos.');
```

END;

3. Verificar el comportamiento en los consumidores. Si ejecutas los bloques de desencolado (DEQUEUE) que usamos antes para cada cliente, notarás el comportamiento selectivo:

CLIENTE_A: Podrá desencolar el mensaje con éxito.

CLIENTE_C: También podrá desencolar el mensaje con éxito.

CLIENTE_B: Si intenta ejecutar su desencolado, el sistema arrojará la excepción de "No hay mensajes disponibles en la cola" (Error ORA-25228), ya que Oracle lo excluyó del mapa de distribución de este mensaje en particular.

ALTERNATIVA: ¿CUÁNDO USAR REGLAS DE FILTRADO (SUBSCRIBER RULES)?

Si prefieres que el emisor no tenga que conocer la lista de clientes, sino que cada cliente decida qué recibir basándose en los datos del mensaje, puedes añadir suscriptores con condiciones utilizando la sintaxis de SQL

```

DECLARE
--
v_suscriptor SYS.AQ$_AGENT;
--
BEGIN
--
v_suscriptor := SYS.AQ$_AGENT('CLIENTE_A', NULL, NULL);
--
```

```
-- El CLIENTE_A solo quiere notificaciones cuyo ID sea mayor a 500
DBMS_AQADM.ADD_SUBSCRIBER(
    queue_name => 'cola_difusion',
    subscriber => v_suscriptor,
    rule       => 'tab.user_data.id_notificacion > 500' -- Condición basada
en el objeto
);
END;
```

Con este enfoque, cuando envías cualquier mensaje, Oracle AQ lee la propiedad dentro del objeto (user_data) y evalúa automáticamente si el suscriptor califica para recibirlo.